STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC-26-0502	Inception.

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (**SMPS**)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

TASK 1: HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

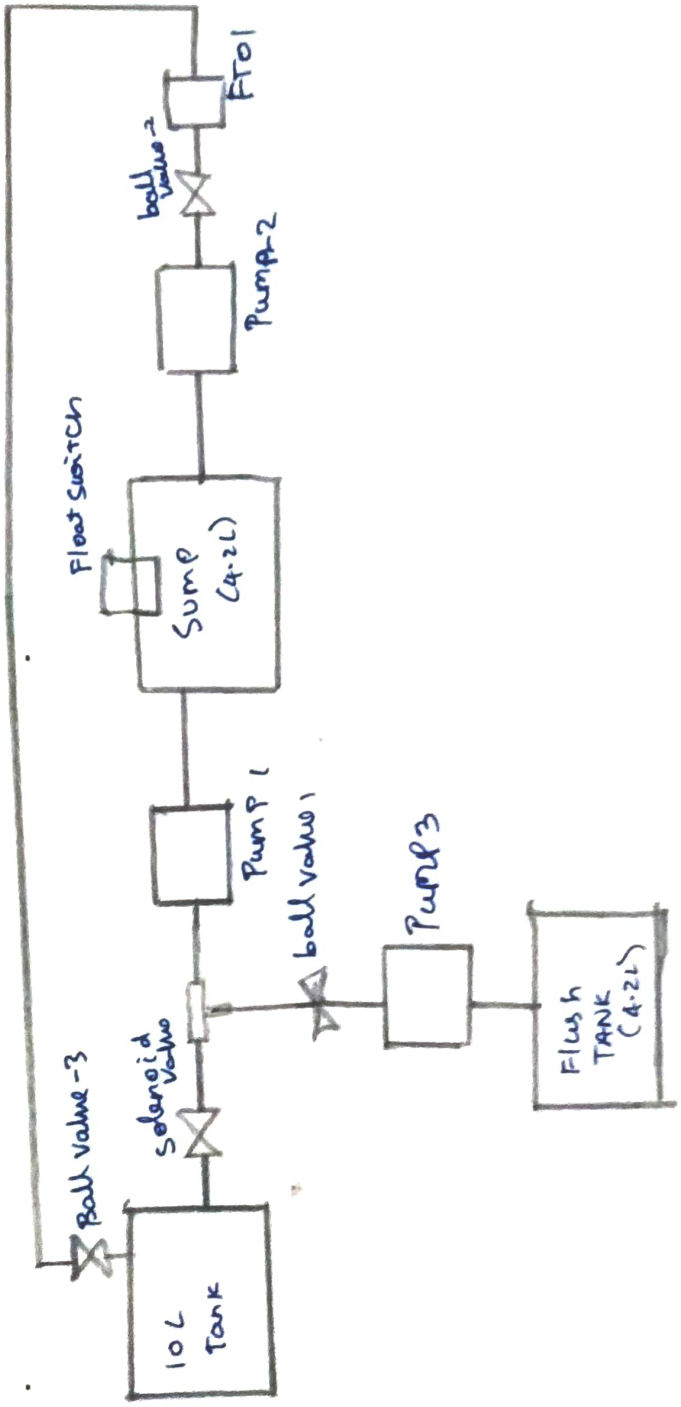
Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

P&ID SKETCH / WORKING SPACE



Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

$D = 8mm = 0.008m$

$A = \pi D^2/4 = \pi(0.008)^2/4 = 5.027 \times 10^{-5} m^2$

$Q\ (m^3/s) = Q\ (L/min) \div 60000$, $V = Q/A$

Target flow	Q (m^3/s)	Theoretical V (m/s)	Actual Q (L/min)	Actual V (m/s)
1 L/min	1.667×10^{-5}	0.332	0.91	0.303
3 L/min	5.000×10^{-5}	0.995	2.31	0.765
6 L/min	1.000×10^{-4}	1.989	2.31	0.765

Theoretical velocity

1 L/min: $Q\ (m^3/s) = 1 \div 60000 = 1.667 \times 10^{-5} m^3/s$

$V = Q/A = 1.667 \times 10^{-5} / 5.027 \times 10^{-5} = 0.332 m/s$

3 L/min: $Q\ (m^3/s) = 3 \div 60000 = 5.000 \times 10^{-5} m^3/s$

$V = Q/A = 5.000 \times 10^{-5} / 5.027 \times 10^{-5} = 0.995 m/s$

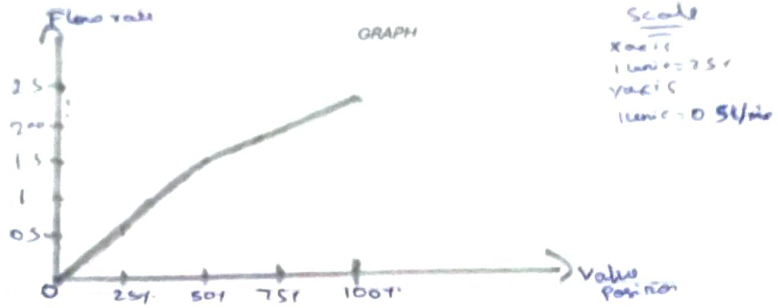
6 L/min: $Q\ (m^3/s) = 6 \div 60000 = 1.000 \times 10^{-4}$

$V = 1.000 \times 10^{-4} \div 5.027 \times 10^{-5} = 1.989 m/s$

TASK 2: FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%, 50%, 75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.56
50%	1.54
70%	1.87
100%	2.31



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 = 0.957

a = 0.01075 L/min Per Hz

b = −0.0116 L/min

Maximum calibration error = 0.164 L/min (10.6% at 50% PWM. Maximum 12.9% at 25%)

Average calibration error = 0.109 L/min (about 9.5%)

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	1710	28.5	60.0	0.266	0.56
2	2083	24.2	86.0	0.383	0.95
3	4359	30.9	141.0	0.824	1.60
4	3606	25.6	141.0	0.683	1.60
5	4366	23.7	184.0	0.738	1.87

CALCULATIONS / OBSERVATIONS

$f = \frac{N}{t}$, $Q = \frac{V}{t} \times 60$

Trial 1:

$N = 1710 \quad t = 28.5s \quad V = 0.266L$

$f = \frac{N}{t} = \frac{1710}{28.5} = 60.00Hz$

$Q = \frac{V}{t} \times 60 = \frac{0.266}{28.5} \times 60 = 0.56L/min$

Trial 2: $N = 2083 \quad t = 24.2s \quad V = 0.383L$

$f = \frac{2083}{24.2} = 86.07Hz$

$Q = \frac{0.383}{24.2} \times 60 = 0.95L/min$

Trial 3:

$N = 4359 \quad t = 30.9s \quad V = 0.824L$

$f = \frac{4359}{30.9} = 141.07Hz$

$Q = \frac{0.824}{30.9} \times 60 = 1.60L/min$

Trial 4

$N = 3606 \quad t = 25.6s \quad V = 0.683L$

$f = \frac{3606}{25.6} = 140.86Hz$

$Q = \frac{0.683}{25.6} \times 60 = 1.60L/min$

Trial 5

$N = 4366 \quad t = 23.7s \quad V = 0.738L$

$f = \frac{4366}{23.7} = 184.22Hz$

$Q = \frac{0.738}{23.7} \times 60 = 1.87L/min$

TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions.

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

1. It represents the average time that a volume of liquid remains inside the tubing section before flowing out. Here, the residence time is 12 seconds

2. $T_r = V/q$ $\quad T_r = \frac{1.0}{2.5} \times 60 = 24$ seconds

The residence time doubles

3. $T_r = \frac{0.5}{50} \times 6.0 = 6$ seconds

The residence time reduces

4. It helps us understand how quickly a change in flow can affect the liquid inside the tubing and reach a downstream sensor. A sudden change in the sensor reading may be delayed by the time required for the liquid to travel through the tubing. It used for identifying whether a measured change is a immediate system response or delayed response.

$$T_r = V/q = \frac{0.5}{2.5} = 0.2 \text{ min}$$

$$T_r = 0.2 \times 60 = 12 \text{ seconds}$$

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

Condition A → Normal operation: The Pump is running, The valve is open, both the Pulse signal and measured flow are normal. No fault is indicated

Condition B → Valve restriction → The Partially closed valve increases resistance to flow, Consequently, the flow rate and FT01 Pulse frequency decrease.

condition-C → Zero flow or Sensor failure → The Pump is ON, but the sensor detects no Pulses and measured flow is zero. Possible cases are failed Pump, blocked tubing an empty source tank, air lock or a failed FT01 sensor or signal wire.

Condition-D → Pump Off: zero flow and zero Pulses are expected when Pump is stopped This is not a fault

condition-E → unstable flow or sensor signal: Intermittent Pulses and fluctuating flow may indicate air in the tubing, inconsistent Pump operation, Partial blockage or a loose electrical connection. which is most difficult to distinguish using FT01 alone Condition C. A zero pulse signal mean either water is not flowing or FT01 is not detecting. Possibilities check are whether water is moving through Pipe, tubing for blockages etc..

HARDWARE LIST

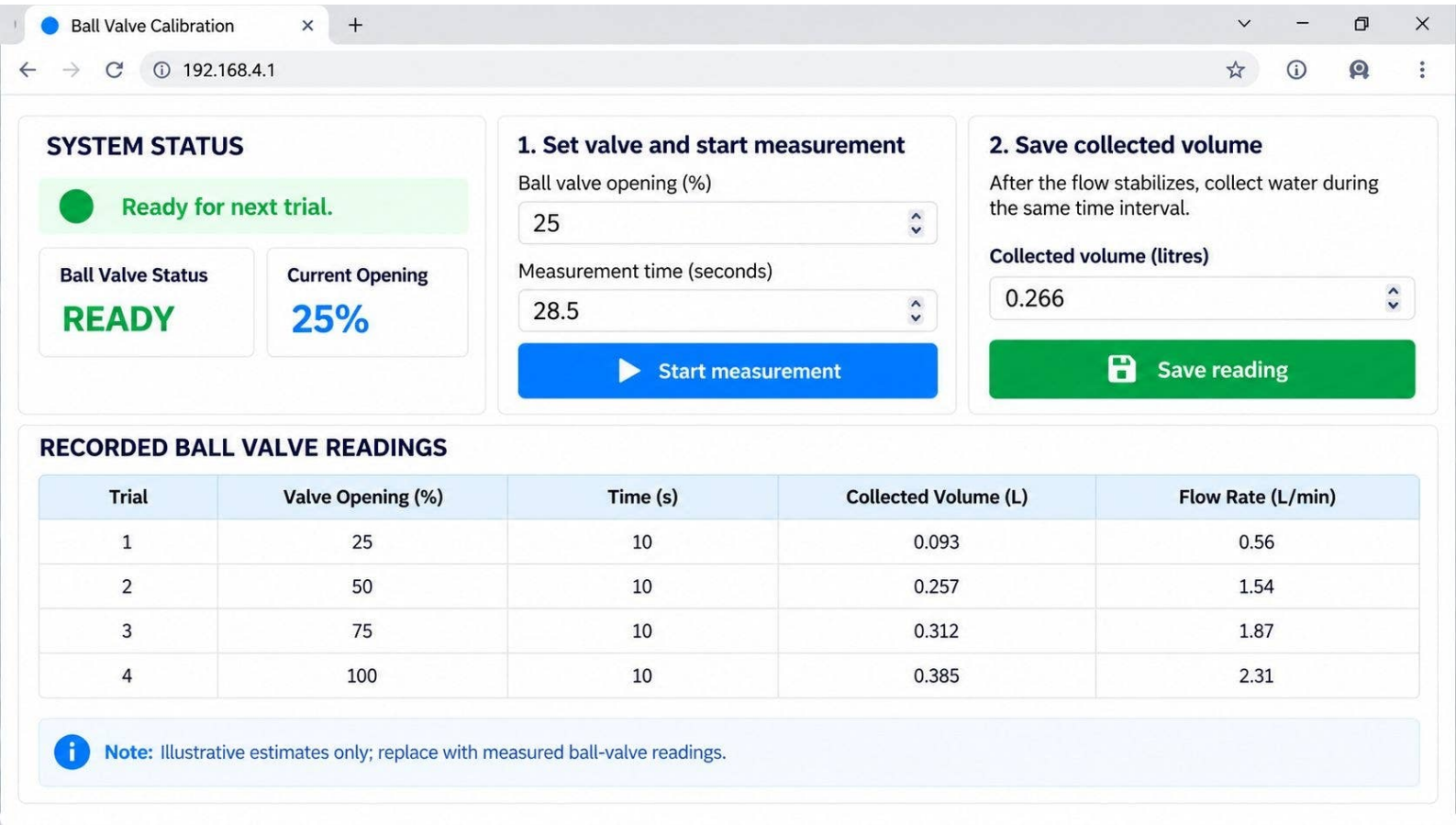
Mechanical Hardware

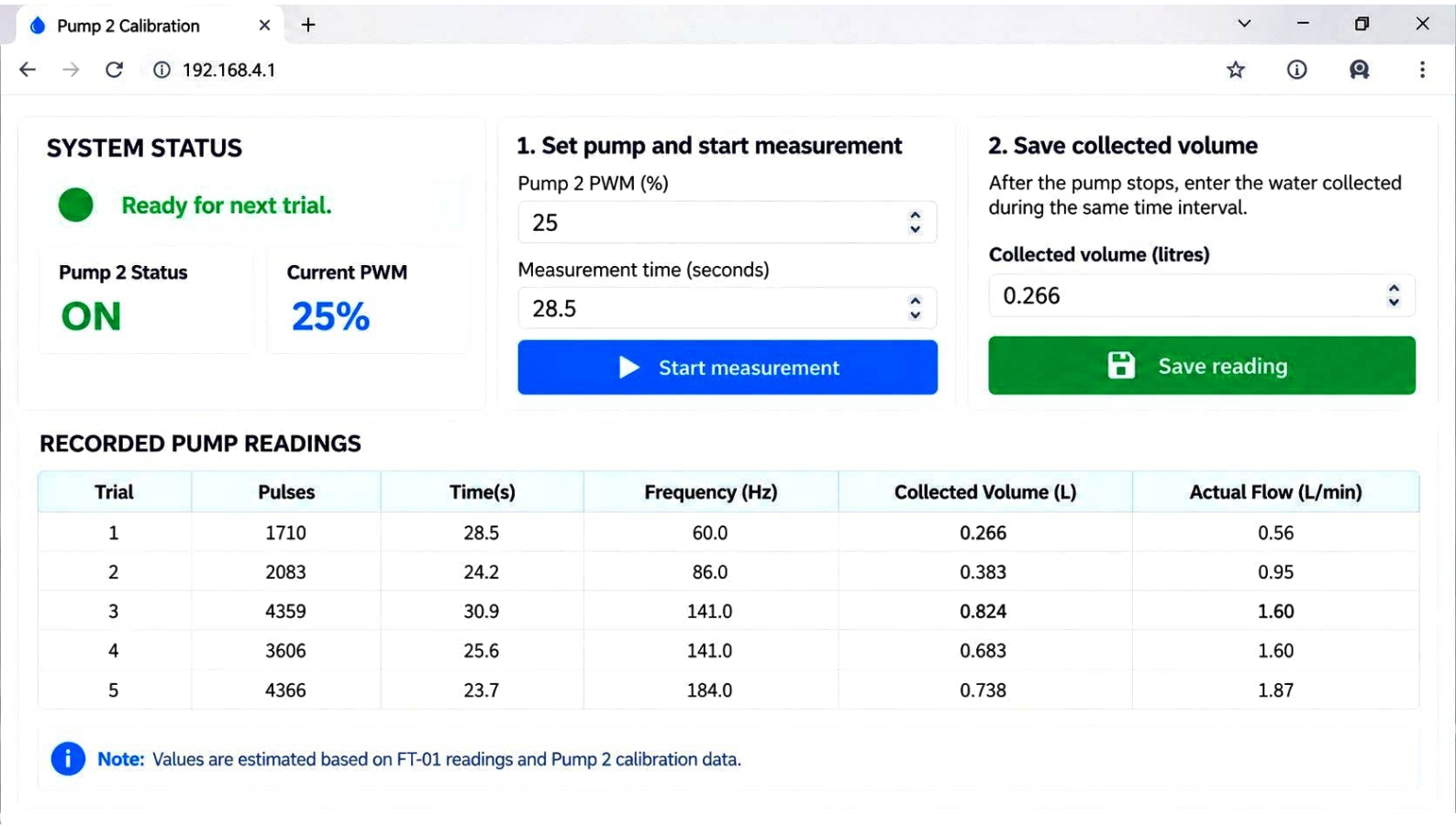
Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe**. Mainly used for Pumps
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line**.

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces**.
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller**. It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board


Ball Valve Calibration
192.168.4.1
SYSTEM STATUS
Ready for next trial.
Ball Valve Status
READY
Current Opening
25%
1. Set valve and start measurement
Ball valve opening (%)
25
Measurement time (seconds)
28.5
Start measurement
2. Save collected volume
After the flow stabilizes, collect water during the same time interval.
Collected volume (litres)
0.266
Save reading
RECORDED BALL VALVE READINGS
Trial | Valve Opening (%) | Time (s) | Collected Volume (L) | Flow Rate (L/min)
1 | 25 | 10 | 0.093 | 0.56
2 | 50 | 10 | 0.257 | 1.54
3 | 75 | 10 | 0.312 | 1.87
4 | 100 | 10 | 0.385 | 2.31
Note: Illustrative estimates only; replace with measured ball-valve readings.


Pump 2 Calibration
192.168.4.1
SYSTEM STATUS
Ready for next trial.
Pump 2 Status
ON
Current PWM
25%
1. Set pump and start measurement
Pump 2 PWM (%)
25
Measurement time (seconds)
28.5
Start measurement
2. Save collected volume
After the pump stops, enter the water collected during the same time interval.
Collected volume (litres)
0.266
Save reading
RECORDED PUMP READINGS
Trial | Pulses | Time(s) | Frequency (Hz) | Collected Volume (L) | Actual Flow (L/min)
1 | 1710 | 28.5 | 60.0 | 0.266 | 0.56
2 | 2083 | 24.2 | 86.0 | 0.383 | 0.95
3 | 4359 | 30.9 | 141.0 | 0.824 | 1.60
4 | 3606 | 25.6 | 141.0 | 0.683 | 1.60
5 | 4366 | 23.7 | 184.0 | 0.738 | 1.87
Note: Values are estimated based on FT-01 readings and Pump 2 calibration data.

CALIBRATION CODE

```
#include <WiFi.h>
#include <WebServer.h>
#include <math.h>

// ================================================
// WIFI
// ================================================
const char* WIFI_SSID = "LAM-ESP32";
const char* WIFI_PASSWORD = "12345678";

WebServer server(80);

// ================================================
// PUMP 2 BTS7960 PINS
// Uses LPWM for speed control and keeps RPWM LOW.
// Confirm these against your PCB.
// ================================================
const int P2_RPWM = 19;
const int P2_LPWM = 18;
const int P2_REN  = 17;
const int P2_LEN  = 16;

// FT01
const int FT01_PIN = 14;

// ================================================
// CALIBRATION STORAGE
// ================================================
const int MAX_TRIALS = 20;

struct Trial {
  unsigned long pulses;
  float timeSeconds;
  float frequencyHz;
  float volumeLitres;
  float actualFlowLpm;
  int pwmPercent;
};

Trial trials[MAX_TRIALS];
int trialCount = 0;

// ================================================
// MEASUREMENT STATE
// ================================================
volatile unsigned long pulseCount = 0;

bool pumpOn = false;
bool measuring = false;
```

```
bool pendingVolume = false;

int currentPWM = 0;

unsigned long startMillis = 0;
unsigned long startPulses = 0;
unsigned long durationMs = 10000;

unsigned long pendingPulses = 0;
float pendingTime = 0;
int pendingPWM = 0;

String statusMessage = "Ready. Set PWM and start Trial 1.";

// ================================================
// PULSE COUNTER
// ================================================
void IRAM_ATTR countPulse() {
  pulseCount++;
}

unsigned long getPulseCount() {
  noInterrupts();
  unsigned long value = pulseCount;
  interrupts();
  return value;
}

// ================================================
// PUMP CONTROL
// ================================================
void pump2SetPWM(int percent) {
  percent = constrain(percent, 0, 100);

  if (percent == 0) {
    analogWrite(P2_RPWM, 0);
    analogWrite(P2_LPWM, 0);
    digitalWrite(P2_REN, LOW);
    digitalWrite(P2_LEN, LOW);

    pumpOn = false;
    currentPWM = 0;
    return;
  }

  // One direction only
  digitalWrite(P2_REN, HIGH);
  digitalWrite(P2_LEN, HIGH);

  analogWrite(P2_RPWM, 0);
  analogWrite(P2_LPWM, map(percent, 0, 100, 0, 255));

  currentPWM = percent;
  pumpOn = true;
}

void stopPump() {
```

```
  pump2SetPWM(0);
}

// ================================================
// START MEASUREMENT
// Pump starts and pulses are counted for the same
// interval used to collect the water.
// ================================================
void startTrial() {
  if (measuring || pendingVolume) {
    server.send(409, "text/plain",
          "Finish or save the current trial first.");
    return;
  }

  if (trialCount >= MAX_TRIALS) {
    server.send(409, "text/plain",
          "Trial table full. Download the CSV.");
    return;
  }

  if (!server.hasArg("pwm") ||
      !server.hasArg("duration")) {
    server.send(400, "text/plain",
          "PWM and duration are required.");
    return;
  }

  int pwm = server.arg("pwm").toInt();
  float seconds = server.arg("duration").toFloat();

  if (pwm < 1 || pwm > 100 ||
      seconds < 1 || seconds > 600) {
    server.send(400, "text/plain",
          "PWM must be 1-100; time 1-600 seconds.");
    return;
  }

  durationMs = (unsigned long)(seconds * 1000.0);
  pendingPWM = pwm;

  // Start counting from a known baseline.
  startPulses = getPulseCount();
  startMillis = millis();

  pump2SetPWM(pwm);
  measuring = true;

  statusMessage = "Trial running. Collect all outlet water now.";
  server.send(200, "text/plain", "Trial started.");
}

// ================================================
// FINISH MEASUREMENT
// Pump stops automatically.
// User then enters measured volume.
// ================================================
```

```
void finishTrial() {
  unsigned long endMillis = millis();
  unsigned long endPulses = getPulseCount();

  pendingPulses = endPulses - startPulses;
  pendingTime = (endMillis - startMillis) / 1000.0;

  if (pendingTime <= 0) pendingTime = 0.001;

  stopPump();

  measuring = false;
  pendingVolume = true;

  statusMessage =
     "Pump stopped. Measure collected water and save volume.";
}

// ================================================
// SAVE MEASURED VOLUME
// ================================================
void saveTrial() {
  if (!pendingVolume) {
    server.send(409, "text/plain",
            "No completed trial waiting for volume.");
   return;
  }

  if (!server.hasArg("volume")) {
    server.send(400, "text/plain",
            "Collected volume is required.");
   return;
  }

  float volume = server.arg("volume").toFloat();

  if (volume <= 0 || volume > 100) {
    server.send(400, "text/plain",
            "Enter a valid collected volume in litres.");
   return;
  }

  Trial &t = trials[trialCount];

  t.pulses = pendingPulses;
  t.timeSeconds = pendingTime;
  t.frequencyHz = pendingPulses / pendingTime;
  t.volumeLitres = volume;
  t.actualFlowLpm = 60.0 * volume / pendingTime;
  t.pwmPercent = pendingPWM;

  trialCount++;
  pendingVolume = false;

  statusMessage =
     "Trial saved. Ready for Trial " +
     String(trialCount + 1) + ".";
```

```
  server.send(200, "text/plain", "Trial saved.");
}

// ================================================
// LINEAR REGRESSION Q = a*f + b
// ================================================
void calculateFit(double &a, double &b,
                  double &r2, double &maxError,
                  double &averageError,
                  double &maxPercent,
                  double &averagePercent) {
  a = b = r2 = maxError = averageError = 0;
  maxPercent = averagePercent = 0;

  if (trialCount < 2) return;

  double sumF = 0, sumQ = 0;
  double sumFF = 0, sumFQ = 0;
  double meanQ = 0;

  for (int i = 0; i < trialCount; i++) {
    double f = trials[i].frequencyHz;
    double q = trials[i].actualFlowLpm;

    sumF += f;
    sumQ += q;
    sumFF += f * f;
    sumFQ += f * q;
    meanQ += q;
  }

  double n = trialCount;
  double denominator = n * sumFF - sumF * sumF;

  if (fabs(denominator) < 1e-12) return;

  a = (n * sumFQ - sumF * sumQ) / denominator;
  b = (sumQ - a * sumF) / n;
  meanQ /= n;

  double ssResidual = 0;
  double ssTotal = 0;
  double sumAbsoluteError = 0;
  double sumPercentError = 0;
  int percentCount = 0;

  for (int i = 0; i < trialCount; i++) {
    double actual = trials[i].actualFlowLpm;
    double predicted = a * trials[i].frequencyHz + b;
    double error = fabs(actual - predicted);

    sumAbsoluteError += error;
    if (error > maxError) maxError = error;

    if (fabs(actual) > 1e-9) {
      double pe = 100.0 * error / fabs(actual);
```

```
    sumPercentError += pe;
    percentCount++;
    if (pe > maxPercent) maxPercent = pe;
   }

   ssResidual += (actual - predicted) *
             (actual - predicted);

   ssTotal += (actual - meanQ) * (actual - meanQ);
  }

  averageError = sumAbsoluteError / n;

  if (percentCount > 0)
   averagePercent = sumPercentError / percentCount;

  if (ssTotal > 1e-12)
   r2 = 1.0 - ssResidual / ssTotal;
  else
   r2 = (ssResidual < 1e-12) ? 1.0 : 0.0;
}

// ================================================
// CSV DOWNLOAD
// ================================================
void downloadCSV() {
  double a, b, r2, maxE, avgE, maxP, avgP;
  calculateFit(a, b, r2, maxE, avgE, maxP, avgP);

  String csv =
     "Trial,PWM_percent,Pulses,Time_s,Frequency_Hz,"
     "Collected_Volume_L,Actual_Flow_L_min,"
     "Predicted_Flow_L_min,Absolute_Error_L_min\n";

  for (int i = 0; i < trialCount; i++) {
   Trial &t = trials[i];
   double predicted =
      (trialCount >= 2) ?
      a * t.frequencyHz + b : NAN;

   csv += String(i + 1) + ",";
   csv += String(t.pwmPercent) + ",";
   csv += String(t.pulses) + ",";
   csv += String(t.timeSeconds, 4) + ",";
   csv += String(t.frequencyHz, 5) + ",";
   csv += String(t.volumeLitres, 4) + ",";
   csv += String(t.actualFlowLpm, 5) + ",";
   csv += (isnan(predicted) ? "" :
         String(predicted, 5)) + ",";
   csv += (isnan(predicted) ? "" :
         String(fabs(t.actualFlowLpm - predicted), 5));
   csv += "\n";
  }

  csv += "\nR_squared," + String(r2, 6) + "\n";
  csv += "Slope_a," + String(a, 8) + "\n";
  csv += "Offset_b," + String(b, 8) + "\n";
```

```
  csv += "Maximum_absolute_error_L_min," +
       String(maxE, 6) + "\n";
  csv += "Average_absolute_error_L_min," +
       String(avgE, 6) + "\n";
  csv += "Maximum_error_percent," +
       String(maxP, 4) + "\n";
  csv += "Average_error_percent," +
       String(avgP, 4) + "\n";

  server.sendHeader("Content-Disposition",
                "attachment; filename=Pump2_FT01_Q23.csv");
  server.send(200, "text/csv", csv);
}

// ================================================
// CLEAR DATA
// ================================================
void clearTrials() {
  if (measuring || pendingVolume) {
    server.send(409, "text/plain",
            "Finish or save the current trial first.");
    return;
  }

  trialCount = 0;
  statusMessage = "Table cleared. Ready for Trial 1.";
  server.send(200, "text/plain", "OK");
}

// ================================================
// WIFI DASHBOARD
// ================================================
const char PAGE[] PROGMEM = R"HTML(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Pump 2 FT01 Calibration</title>
<style>
body{margin:0;background:#05090f;color:#e6f7ff;font-family:Arial}
header{background:#071b2c;padding:18px;border-bottom:2px solid #00b7ff}
h2{color:#52d6ff;margin:0}
main{padding:12px;max-width:1100px;margin:auto}
.card{background:#0a1522;border:1px solid #173d55;border-radius:12px;padding:15px;margin-
bottom:12px}
.grid{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:12px}
label{display:block;color:#9ab9cc;font-size:13px;margin:10px 0 5px}
input{box-sizing:border-box;width:100%;padding:11px;background:#06101a;color:white;border:1px
solid #24516c;border-radius:7px}
button,a{display:inline-block;margin:8px 4px 0 0;padding:11px
14px;background:#007eae;color:white;border:0;border-radius:7px;text-decoration:none;font-
weight:bold}
.value{font-size:23px;color:#52d6ff;font-weight:bold}
.small{color:#9ab9cc;font-size:12px}
.scroll{overflow-x:auto}
table{width:100%;border-collapse:collapse;font-size:12px}
th,td{padding:9px 5px;border-bottom:1px solid #203748;text-align:center}
```

```
th{color:#52d6ff}
@media(max-width:600px){.grid{grid-template-columns:1fr}table{font-size:10px}}
</style>
</head>
<body>
<header>
<h2>◉ PUMP 2 — FT01 CALIBRATION</h2>
<p class="small">Q2.3 • Flow measurement • Wi-Fi dashboard</p>
</header>
<main>

<div class="card">
<div class="small">SYSTEM STATUS</div>
<div class="value" id="status">Connecting...</div>
<p class="small" id="detail"></p>
<div class="grid">
  <div><label>Pump 2 status</label><div class="value" id="pump">OFF</div></div>
  <div><label>Current PWM</label><div class="value" id="pwmNow">0%</div></div>
  <div><label>Pulses in current trial</label><div class="value" id="pulses">0</div></div>
  <div><label>Pulse frequency</label><div class="value"><span id="freq">0</span> Hz</div></div>
</div>
</div>

<div class="card">
<h3>1. Set pump and start measurement</h3>
<label>Pump 2 PWM (%)</label>
<input id="pwm" type="number" min="1" max="100" value="25">
<label>Measurement time (seconds)</label>
<input id="duration" type="number" min="1" max="600" value="10">
<p class="small">Start collecting water at the outlet when the trial begins. Pump 2 stops
automatically when the timer ends.</p>
<button id="start" onclick="startTrial()">Start trial</button>
</div>

<div class="card">
<h3>2. Save collected volume</h3>
<p class="small">After the pump stops, measure the water collected during the same timed
interval.</p>
<label>Collected volume (litres)</label>
<input id="volume" type="number" min="0.001" step="0.001" placeholder="Enter measured litres">
<button id="save" onclick="saveTrial()">Save trial</button>
<a href="/csv">Download CSV</a>
<button onclick="clearTrials()">Clear table</button>
</div>

<div class="card">
<h3>Q2.3 experimental data</h3>
<div class="scroll">
<table>
<thead><tr>
<th>Trial</th><th>PWM %</th><th>Pulses</th><th>Time (s)</th>
<th>Frequency (Hz)</th><th>Volume (L)</th><th>Actual Q (L/min)</th>
</tr></thead>
<tbody id="rows"></tbody>
</table>
</div>
<p class="small">f = N/t; actual Q = 60V/t.</p>
```

```
</div>

<div class="card">
<h3>Calibration equation: Q = af + b</h3>
<div class="grid">
<div><label>R²</label><div class="value" id="r2">--</div></div>
<div><label>Slope a</label><div class="value" id="a">--</div></div>
<div><label>Offset b</label><div class="value" id="b">--</div></div>
<div><label>Max absolute error (L/min)</label><div class="value" id="maxE">--</div></div>
<div><label>Average absolute error (L/min)</label><div class="value" id="avgE">--</div></div>
<div><label>Max / average error (%)</label><div class="value" id="percentE">--</div></div>
</div>
<p class="small" id="equation">Record at least two different flow conditions to calculate the fit.</p>
</div>

</main>
<script>
async function startTrial(){
  const pwm=Number(document.getElementById('pwm').value);
  const duration=Number(document.getElementById('duration').value);
  if(pwm<1||pwm>100||duration<1||duration>600){
    alert('Enter PWM 1-100 and duration 1-600 seconds.');return;
  }
  const r=await fetch('/start?pwm='+pwm+'&duration='+duration);
  if(!r.ok)alert(await r.text());
  refresh();
}

async function saveTrial(){
  const volume=Number(document.getElementById('volume').value);
  if(volume<=0){alert('Enter the actual collected volume in litres.');return;}
  const r=await fetch('/save?volume='+encodeURIComponent(volume));
  if(!r.ok){alert(await r.text());return;}
  document.getElementById('volume').value='';
  refresh();
}

async function clearTrials(){
  if(!confirm('Clear all saved trials?'))return;
  const r=await fetch('/clear');
  if(!r.ok)alert(await r.text());
  refresh();
}

async function refresh(){
 try{
  const r=await fetch('/data');
  const d=await r.json();
  document.getElementById('status').textContent=d.status;
  document.getElementById('detail').textContent=d.detail;
  document.getElementById('pump').textContent=d.pump?'ON':'OFF';
  document.getElementById('pwmNow').textContent=d.pwm+'%';
  document.getElementById('pulses').textContent=d.livePulses;
  document.getElementById('freq').textContent=d.frequency.toFixed(2);
  document.getElementById('start').disabled=d.measuring||d.pending||d.full;
  document.getElementById('save').disabled=!d.pending;
```

```
 let html='';
 d.trials.forEach((t,i)=>{
  html+='<tr><td>'+(i+1)+'</td><td>'+t.pwm+'</td><td>'+
   t.pulses+'</td><td>'+t.time.toFixed(3)+'</td><td>'+
   t.frequency.toFixed(3)+'</td><td>'+t.volume.toFixed(3)+
   '</td><td>'+t.flow.toFixed(3)+'</td></tr>';
 });
 document.getElementById('rows').innerHTML=html;

 document.getElementById('r2').textContent=d.n<2?'Need 2+ trials':d.r2.toFixed(5);
 document.getElementById('a').textContent=d.n<2?'--':d.a.toFixed(6);
 document.getElementById('b').textContent=d.n<2?'--':d.b.toFixed(6);
 document.getElementById('maxE').textContent=d.n<2?'--':d.maxE.toFixed(4);
 document.getElementById('avgE').textContent=d.n<2?'--':d.avgE.toFixed(4);
 document.getElementById('percentE').textContent=d.n<2?'--':
   d.maxP.toFixed(2)+'% / '+d.avgP.toFixed(2)+'%';
 document.getElementById('equation').textContent=d.n<2?
   'Record at least two different flow conditions to calculate the fit.':
   'Q = '+d.a.toFixed(5)+' f '+(d.b<0?'- ':'+ ')+Math.abs(d.b).toFixed(5);
 }catch(e){}
}
setInterval(refresh,500);
refresh();
</script>
</body>
</html>
)HTML";

// ================================================
// DASHBOARD JSON
// ================================================
void sendData() {
  unsigned long now = millis();
  unsigned long currentPulses = getPulseCount();

  unsigned long livePulses = measuring
     ? currentPulses - startPulses : 0;

  float elapsed = measuring
     ? (now - startMillis) / 1000.0 : 0;

  float frequency = (measuring && elapsed > 0)
     ? livePulses / elapsed : 0;

  unsigned long remaining = 0;

  if (measuring) {
    unsigned long passed = now - startMillis;
    remaining = passed >= durationMs ? 0 :
            (durationMs - passed + 999) / 1000;
  }

  double a,b,r2,maxE,avgE,maxP,avgP;
  calculateFit(a,b,r2,maxE,avgE,maxP,avgP);

  String detail;
```

```
  if (measuring)
    detail = "Measuring. Time remaining: " + String(remaining) + " s";
  else if (pendingVolume)
    detail = "Measurement complete. Enter volume and save.";
  else
    detail = "Ready for next trial.";

  String json = "{";
  json += "\"status\":\"" + statusMessage + "\",";
  json += "\"detail\":\"" + detail + "\",";
  json += "\"pump\":" + String(pumpOn ? "true" : "false") + ",";
  json += "\"pwm\":" + String(currentPWM) + ",";
  json += "\"measuring\":" + String(measuring ? "true" : "false") + ",";
  json += "\"pending\":" + String(pendingVolume ? "true" : "false") + ",";
  json += "\"full\":" + String(trialCount >= MAX_TRIALS ? "true" : "false") + ",";
  json += "\"livePulses\":" + String(livePulses) + ",";
  json += "\"frequency\":" + String(frequency, 5) + ",";
  json += "\"n\":" + String(trialCount) + ",";
  json += "\"a\":" + String(a, 8) + ",";
  json += "\"b\":" + String(b, 8) + ",";
  json += "\"r2\":" + String(r2, 8) + ",";
  json += "\"maxE\":" + String(maxE, 8) + ",";
  json += "\"avgE\":" + String(avgE, 8) + ",";
  json += "\"maxP\":" + String(maxP, 8) + ",";
  json += "\"avgP\":" + String(avgP, 8) + ",";
  json += "\"trials\":[";

  for (int i=0; i<trialCount; i++) {
    if (i) json += ",";
    Trial &t=trials[i];

    json += "{";
    json += "\"pwm\":" + String(t.pwmPercent) + ",";
    json += "\"pulses\":" + String(t.pulses) + ",";
    json += "\"time\":" + String(t.timeSeconds, 5) + ",";
    json += "\"frequency\":" + String(t.frequencyHz, 6) + ",";
    json += "\"volume\":" + String(t.volumeLitres, 5) + ",";
    json += "\"flow\":" + String(t.actualFlowLpm, 6);
    json += "}";
  }

  json += "]}";
  server.send(200, "application/json", json);
}

// ================================================
// SETUP
// ================================================
void setup() {
  Serial.begin(115200);

  pinMode(P2_RPWM, OUTPUT);
  pinMode(P2_LPWM, OUTPUT);
  pinMode(P2_REN, OUTPUT);
  pinMode(P2_LEN, OUTPUT);

  stopPump();
```

```
  // FT01 must provide a safe 3.3 V logic signal.
  pinMode(FT01_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FT01_PIN),
                  countPulse, RISING);

  WiFi.mode(WIFI_AP);
  WiFi.softAP(WIFI_SSID, WIFI_PASSWORD);

  server.on("/", []() {
    server.send_P(200, "text/html", PAGE);
  });

  server.on("/start", startTrial);
  server.on("/save", saveTrial);
  server.on("/data", sendData);
  server.on("/csv", downloadCSV);
  server.on("/clear", clearTrials);

  server.begin();

  Serial.println();
  Serial.println("PUMP 2 FT01 CALIBRATION READY");
  Serial.print("Wi-Fi: ");
  Serial.println(WIFI_SSID);
  Serial.print("Dashboard: http://");
  Serial.println(WiFi.softAPIP());
}

// ================================================
// MAIN LOOP
// ================================================
void loop() {
  server.handleClient();

  if (measuring &&
      millis() - startMillis >= durationMs) {
    finishTrial();
  }
}
```

WORKING CODE

```
#include <WiFi.h>
#include <WebServer.h>

// ======================================================
// WIFI
// ======================================================
const char* AP_SSID = "LAM-ESP32";
```

```
const char* AP_PASSWORD = "12345678";

WebServer server(80);

// ======================================================
// GPIO PIN ASSIGNMENTS
// Confirm these against your actual PCB before powering.
// ======================================================

// Pump 1 BTS7960
const int P1_RPWM = 33;
const int P1_LPWM = 32;
const int P1_REN  = 26;
const int P1_LEN  = 13;

// Pump 2 BTS7960
const int P2_RPWM = 19;
const int P2_LPWM = 18;
const int P2_REN  = 17;
const int P2_LEN  = 16;

// Pump 3 BTS7960
const int P3_RPWM = 23;
const int P3_LPWM = 25;
const int P3_REN  = 21;
const int P3_LEN  = 22;

// Sensors and control
const int FT01_PIN = 14;
const int FLOAT_PIN = 35;
const int START_STOP_PIN = 4;
const int SOLENOID_PIN = 27;

// ======================================================
// PWM SETTINGS
// ======================================================
const int CASE1_P1_PWM = 50;
const int CASE1_P2_PWM = 100;

const int CASE2_P1_PWM = 50;
const int CASE2_P2_PWM = 25;
const int CASE2_P3_PWM = 80;

// ======================================================
// CALIBRATION
// Flow rates read from your handwritten measurements.
// ======================================================
const float P1_FLOW_25  = 0.41;
const float P1_FLOW_50  = 1.09;
const float P1_FLOW_75  = 1.62;
const float P1_FLOW_100 = 2.00;

const float P2_FLOW_25  = 0.56;
const float P2_FLOW_50  = 1.54;
const float P2_FLOW_75  = 1.87;
const float P2_FLOW_100 = 2.31;
```

```
// Pump 3: using the more consistent 75% and 100% readings.
// 75% = 1.71 L/min; 100% = 2.14 L/min.
const float P3_FLOW_75  = 1.71;
const float P3_FLOW_100 = 2.14;

// IMPORTANT:
// Replace this placeholder with your measured FT01 pulses/L.
// Collect a known volume, count pulses, then calculate:
// pulses per litre = pulses counted / litres collected.
const float FT01_PULSES_PER_LITRE = 1200.0;

// Case 1 volume target
const float CASE1_TARGET_LITRES = 7.0;

// Case 2 estimated flush volume
const float FLUSH_TARGET_LITRES = 4.0;

// Timing
const unsigned long PUMP2_START_DELAY_MS = 60000UL;
const unsigned long NO_FLOW_TIMEOUT_MS = 5000UL;
const unsigned long CASE2_START_COUNTDOWN_MS = 5000UL;
const unsigned long SHUTDOWN_COUNTDOWN_MS = 10000UL;

// FT01 flow update interval
const unsigned long FLOW_UPDATE_MS = 1000UL;

// ====================================================
// STATE MACHINE
// ====================================================
enum SystemState {
  SYSTEM_OFF,
  CASE1_WAIT_PUMP2,
  CASE1_RUNNING,
  CASE1_WAIT_NO_FLOW,
  CASE2_COUNTDOWN,
  CASE2_RUNNING,
  FINAL_SHUTDOWN
};

SystemState state = SYSTEM_OFF;

// ====================================================
// VARIABLES
// ====================================================
volatile unsigned long ftPulseCount = 0;

unsigned long processStartMs = 0;
unsigned long case2CountdownStartMs = 0;
unsigned long case2StartMs = 0;
unsigned long finalCountdownStartMs = 0;

unsigned long lastFlowUpdateMs = 0;
unsigned long lastPulseSnapshot = 0;
unsigned long lastPulseTimeMs = 0;
unsigned long noFlowStartMs = 0;

float totalLitres = 0.0;
```

```
float flowLpm = 0.0;
float pulsesPerMinute = 0.0;

bool pump1On = false;
bool pump2On = false;
bool pump3On = false;
bool solenoidOpen = false;

bool lastButtonReading = HIGH;
bool stableButtonState = HIGH;
unsigned long lastButtonChangeMs = 0;
const unsigned long BUTTON_DEBOUNCE_MS = 40;

// ======================================================
// FT01 INTERRUPT
// ======================================================
void IRAM_ATTR onFlowPulse() {
  ftPulseCount++;
}

// ======================================================
// PUMP CONTROL
// One direction only: opposite PWM pin stays LOW.
// ======================================================
void configurePump(int rpwm, int lpwm, int ren, int len) {
  pinMode(rpwm, OUTPUT);
  pinMode(lpwm, OUTPUT);
  pinMode(ren, OUTPUT);
  pinMode(len, OUTPUT);

  digitalWrite(ren, LOW);
  digitalWrite(len, LOW);
  analogWrite(rpwm, 0);
  analogWrite(lpwm, 0);
}

void setPump(int rpwm, int lpwm, int ren, int len,
          int pwmPercent, bool &status) {
  pwmPercent = constrain(pwmPercent, 0, 100);

  if (pwmPercent == 0) {
    analogWrite(rpwm, 0);
    analogWrite(lpwm, 0);
    digitalWrite(ren, LOW);
    digitalWrite(len, LOW);
    status = false;
    return;
  }

  digitalWrite(ren, HIGH);
  digitalWrite(len, HIGH);

  analogWrite(lpwm, 0);
  analogWrite(rpwm, map(pwmPercent, 0, 100, 0, 255));

  status = true;
}
```

```
void pump1Start(int pwm) {
  setPump(P1_RPWM, P1_LPWM, P1_REN, P1_LEN,
        pwm, pump1On);
}

void pump1Stop() {
  setPump(P1_RPWM, P1_LPWM, P1_REN, P1_LEN,
        0, pump1On);
}

void pump2Start(int pwm) {
  setPump(P2_RPWM, P2_LPWM, P2_REN, P2_LEN,
        pwm, pump2On);
}

void pump2Stop() {
  setPump(P2_RPWM, P2_LPWM, P2_REN, P2_LEN,
        0, pump2On);
}

void pump3Start(int pwm) {
  setPump(P3_RPWM, P3_LPWM, P3_REN, P3_LEN,
        pwm, pump3On);
}

void pump3Stop() {
  setPump(P3_RPWM, P3_LPWM, P3_REN, P3_LEN,
        0, pump3On);
}

void solenoidSet(bool open) {
  // Assumes HIGH = valve open.
  // Verify this with your relay/driver before operation.
  digitalWrite(SOLENOID_PIN, open ? HIGH : LOW);
  solenoidOpen = open;
}

void allOutputsOff() {
  pump1Stop();
  pump2Stop();
  pump3Stop();
  solenoidSet(false);
}

// ======================================================
// FLOW CALCULATIONS
// ======================================================
unsigned long getPulseCount() {
  noInterrupts();
  unsigned long count = ftPulseCount;
  interrupts();
  return count;
}

void updateFlowMeasurements() {
  unsigned long now = millis();
```

```
  if (now - lastFlowUpdateMs < FLOW_UPDATE_MS) {
    return;
  }

  unsigned long currentPulses = getPulseCount();
  unsigned long pulseDifference = currentPulses - lastPulseSnapshot;

  float seconds =
      (now - lastFlowUpdateMs) / 1000.0;

  if (seconds <= 0) seconds = 1.0;

  pulsesPerMinute = pulseDifference * 60.0 / seconds;

  if (FT01_PULSES_PER_LITRE > 0) {
    flowLpm =
        pulsesPerMinute / FT01_PULSES_PER_LITRE;

    totalLitres =
        currentPulses / FT01_PULSES_PER_LITRE;
  } else {
    flowLpm = 0;
    totalLitres = 0;
  }

  if (pulseDifference > 0) {
    lastPulseTimeMs = now;
  }

  lastPulseSnapshot = currentPulses;
  lastFlowUpdateMs = now;
}

// ====================================================
// FLOW RATE ESTIMATION FROM YOUR CALIBRATION
// Linear interpolation between measured points.
// ====================================================
float interpolateFlow(float pwm, float pwmA, float flowA,
                      float pwmB, float flowB) {
  if (pwm <= pwmA) return flowA;
  if (pwm >= pwmB) return flowB;

  return flowA +
      ((pwm - pwmA) / (pwmB - pwmA)) *
      (flowB - flowA);
}

float estimatedPump1Flow(int pwm) {
  if (pwm <= 25)
    return P1_FLOW_25 * pwm / 25.0;

  if (pwm <= 50)
    return interpolateFlow(pwm, 25, P1_FLOW_25,
                           50, P1_FLOW_50);

  if (pwm <= 75)
```

```
  return interpolateFlow(pwm, 50, P1_FLOW_50,
                         75, P1_FLOW_75);

 return interpolateFlow(pwm, 75, P1_FLOW_75,
                        100, P1_FLOW_100);
}

float estimatedPump2Flow(int pwm) {
 if (pwm <= 25)
  return P2_FLOW_25 * pwm / 25.0;

 if (pwm <= 50)
  return interpolateFlow(pwm, 25, P2_FLOW_25,
                         50, P2_FLOW_50);

 if (pwm <= 75)
  return interpolateFlow(pwm, 50, P2_FLOW_50,
                         75, P2_FLOW_75);

 return interpolateFlow(pwm, 75, P2_FLOW_75,
                        100, P2_FLOW_100);
}

float estimatedPump3Flow(int pwm) {
 if (pwm <= 75)
  return P3_FLOW_75 * pwm / 75.0;

 return interpolateFlow(pwm, 75, P3_FLOW_75,
                        100, P3_FLOW_100);
}

// ======================================================
// START / STOP PROCESS
// ======================================================
void startProcess() {
 allOutputsOff();

 noInterrupts();
 ftPulseCount = 0;
 interrupts();

 totalLitres = 0;
 flowLpm = 0;
 pulsesPerMinute = 0;

 lastPulseSnapshot = 0;
 lastPulseTimeMs = millis();
 lastFlowUpdateMs = millis();

 pump1Start(CASE1_P1_PWM);
 solenoidSet(true);

 processStartMs = millis();
 state = CASE1_WAIT_PUMP2;

 Serial.println("SYSTEM STARTED");
 Serial.println("CASE 1: Pump 1 + solenoid started");
```

```
  Serial.println("Pump 2 starts after 60 seconds");
}

void stopProcess() {
  allOutputsOff();
  state = SYSTEM_OFF;

  Serial.println("SYSTEM OFF");
}

void toggleProcess() {
  if (state == SYSTEM_OFF) {
    startProcess();
  } else {
    stopProcess();
  }
}

// ======================================================
// PHYSICAL BUTTON
// Momentary button from GPIO4 to GND.
// ======================================================
void handleButton() {
  bool reading = digitalRead(START_STOP_PIN);
  unsigned long now = millis();

  if (reading != lastButtonReading) {
    lastButtonChangeMs = now;
    lastButtonReading = reading;
  }

  if (now - lastButtonChangeMs > BUTTON_DEBOUNCE_MS) {
    if (reading != stableButtonState) {
      stableButtonState = reading;

      if (stableButtonState == LOW) {
        toggleProcess();
      }
    }
  }
}

// ======================================================
// FLOAT SWITCH
// Assumption from your previous wiring description:
// LOW = float completely UP / sump full.
// HIGH = float down / level needs filling.
//
// GPIO35 has no internal pull-up. Provide the required
// external signal conditioning/pull resistor.
// ======================================================
bool sumpFloatCompletelyUp() {
  return digitalRead(FLOAT_PIN) == LOW;
}

void handleFloatControl() {
  if (state != CASE1_WAIT_PUMP2 &&
```

```
    state != CASE1_RUNNING) {
  return;
 }

 if (sumpFloatCompletelyUp()) {
  pump1Stop();
 } else if (totalLitres < CASE1_TARGET_LITRES) {
  int pwm = CASE1_P1_PWM;
  pump1Start(pwm);
 }
}

// ======================================================
// STATE MACHINE
// ======================================================
void runProcess() {
 unsigned long now = millis();

 switch (state) {
  case SYSTEM_OFF:
   break;

  case CASE1_WAIT_PUMP2: {
   handleFloatControl();

   if (now - processStartMs >= PUMP2_START_DELAY_MS) {
    pump2Start(CASE1_P2_PWM);
    state = CASE1_RUNNING;
    Serial.println("CASE 1: Pump 2 started");
   }
   break;
  }

  case CASE1_RUNNING: {
   handleFloatControl();

   if (totalLitres >= CASE1_TARGET_LITRES) {
    pump1Stop();
    solenoidSet(false);

    noFlowStartMs = 0;
    state = CASE1_WAIT_NO_FLOW;

    Serial.println("7 L reached: Pump 1 OFF");
    Serial.println("Solenoid CLOSED; Pump 2 continues");
    Serial.println("Waiting for FT01 no-flow condition");
   }
   break;
  }

  case CASE1_WAIT_NO_FLOW: {
   // Pump 2 remains ON while waiting.
   pump1Stop();
   solenoidSet(false);

   if (now - lastPulseTimeMs >= NO_FLOW_TIMEOUT_MS) {
    pump2Stop();
```

```
    case2CountdownStartMs = now;
    state = CASE2_COUNTDOWN;

    Serial.println("FT01: no flow for 5 seconds");
    Serial.println("CASE 2 activates in 5 seconds");
   }
   break;
  }

  case CASE2_COUNTDOWN: {
   unsigned long elapsed = now - case2CountdownStartMs;

   if (elapsed >= CASE2_START_COUNTDOWN_MS) {
    // Reset FT01 measurement for the flushing stage.
    noInterrupts();
    ftPulseCount = 0;
    interrupts();

    totalLitres = 0;
    lastPulseSnapshot = 0;
    lastPulseTimeMs = now;
    lastFlowUpdateMs = now;

    pump1Start(CASE2_P1_PWM);
    pump2Start(CASE2_P2_PWM);
    pump3Start(CASE2_P3_PWM);

    case2StartMs = now;
    state = CASE2_RUNNING;

    Serial.println("CASE 2: ALL THREE PUMPS ON");
    Serial.println("P1=50%, P2=25%, P3=80%");
   }
   break;
  }

  case CASE2_RUNNING: {
   float p3Flow = estimatedPump3Flow(CASE2_P3_PWM);

   if (p3Flow <= 0) {
    // Never allow division by zero.
    stopProcess();
    break;
   }

   unsigned long estimatedFlushMs =
      (unsigned long)((FLUSH_TARGET_LITRES / p3Flow)
                * 60000.0);

   unsigned long elapsed = now - case2StartMs;

   if (elapsed >= estimatedFlushMs -
      min(estimatedFlushMs, SHUTDOWN_COUNTDOWN_MS)) {
    finalCountdownStartMs = now;
    state = FINAL_SHUTDOWN;
```

```
      Serial.println("Final shutdown countdown started");
     }
     break;
    }

    case FINAL_SHUTDOWN: {
     if (now - finalCountdownStartMs >=
        SHUTDOWN_COUNTDOWN_MS) {
      stopProcess();
      Serial.println("FLUSH CYCLE COMPLETE");
     }
     break;
    }
  }
}

// ======================================================
// DASHBOARD HELPERS
// ======================================================
String stateName() {
  switch (state) {
    case SYSTEM_OFF: return "SYSTEM OFF";
    case CASE1_WAIT_PUMP2: return "CASE 1 - WAITING FOR PUMP 2";
    case CASE1_RUNNING: return "CASE 1 - MEASUREMENT";
    case CASE1_WAIT_NO_FLOW: return "CASE 1 - WAITING FOR NO FLOW";
    case CASE2_COUNTDOWN: return "CASE 2 ACTIVATES IN";
    case CASE2_RUNNING: return "CASE 2 - FLUSHING";
    case FINAL_SHUTDOWN: return "FINAL SHUTDOWN COUNTDOWN";
  }
  return "UNKNOWN";
}

unsigned long remainingSeconds() {
  unsigned long now = millis();

  if (state == CASE1_WAIT_PUMP2) {
    unsigned long elapsed = now - processStartMs;
    if (elapsed >= PUMP2_START_DELAY_MS) return 0;
    return (PUMP2_START_DELAY_MS - elapsed + 999) / 1000;
  }

  if (state == CASE2_COUNTDOWN) {
    unsigned long elapsed = now - case2CountdownStartMs;
    if (elapsed >= CASE2_START_COUNTDOWN_MS) return 0;
    return (CASE2_START_COUNTDOWN_MS - elapsed + 999) / 1000;
  }

  if (state == CASE2_RUNNING || state == FINAL_SHUTDOWN) {
    float p3Flow = estimatedPump3Flow(CASE2_P3_PWM);
    unsigned long totalMs =
      (unsigned long)((FLUSH_TARGET_LITRES / p3Flow) * 60000.0);

    unsigned long elapsed = now - case2StartMs;
    unsigned long remaining =
      (elapsed >= totalMs) ? 0 : (totalMs - elapsed);

    return (remaining + 999) / 1000;
```

```
  }

  if (state == CASE1_WAIT_NO_FLOW) {
    unsigned long elapsed = now - lastPulseTimeMs;
    if (elapsed >= NO_FLOW_TIMEOUT_MS) return 0;
    return (NO_FLOW_TIMEOUT_MS - elapsed + 999) / 1000;
  }

  return 0;
}

String onOff(bool value) {
  return value ? "ON" : "OFF";
}

String valveText() {
  return solenoidOpen ? "OPEN" : "CLOSED";
}

// ======================================================
// HTML DASHBOARD
// Black background, blue fluid theme.
// ======================================================
const char PAGE[] PROGMEM = R"HTML(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Fluid Mechanics Control</title>
<style>
body {
  margin:0; background:#05090f; color:#e6f7ff;
  font-family:Arial,sans-serif;
}
header {
  padding:18px; background:#071b2c;
  border-bottom:2px solid #00b7ff;
}
h1 {font-size:22px;margin:0;color:#52d6ff}
.sub {color:#8caec3;font-size:12px;margin-top:7px}
main {padding:12px;max-width:1000px;margin:auto}
.grid {display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:10px}
.card {
  background:#0a1522;border:1px solid #173d55;
  border-radius:12px;padding:14px;min-width:0;
}
.wide {grid-column:1/-1}
.label {font-size:12px;color:#8caec3}
.value {font-size:22px;font-weight:bold;margin-top:7px}
.small {font-size:13px;color:#b8d6e6;margin-top:8px}
.blue {color:#52d6ff}
.green {color:#65f0ad}
.red {color:#ff7f89}
.orange {color:#ffc66d}
.bar {height:7px;background:#182a3a;border-radius:6px;overflow:hidden;margin-top:10px}
.fill {height:100%;background:#00b7ff;width:0%}
.pump {border-left:3px solid #00b7ff}
```

```
footer {color:#6f8da1;font-size:11px;padding:18px;text-align:center}
@media(max-width:480px){.grid{grid-template-columns:1fr}.wide{grid-column:auto}}
</style>
</head>
<body>
<header>
<h1>◉ FLUID MECHANICS CONTROL</h1>
<div class="sub">ESP32 • FLOW MONITORING • CASE 1 / CASE 2</div>
</header>
<main>
<div class="grid">
 <div class="card wide">
  <div class="label">SYSTEM STATUS</div>
  <div class="value blue" id="state">Connecting...</div>
  <div class="small">Stage timer: <b id="timer">--</b> seconds</div>
  <div class="bar"><div class="fill" id="progress"></div></div>
 </div>

 <div class="card pump">
  <div class="label">PUMP 1</div>
  <div class="value" id="p1">--</div>
  <div class="small">PWM: <b id="p1pwm">--</b>%</div>
  <div class="small">Estimated flow: <b id="p1flow">--</b> L/min</div>
 </div>

 <div class="card pump">
  <div class="label">PUMP 2</div>
  <div class="value" id="p2">--</div>
  <div class="small">PWM: <b id="p2pwm">--</b>%</div>
  <div class="small">Estimated flow: <b id="p2flow">--</b> L/min</div>
 </div>

 <div class="card pump">
  <div class="label">PUMP 3</div>
  <div class="value" id="p3">--</div>
  <div class="small">PWM: <b id="p3pwm">--</b>%</div>
  <div class="small">Estimated flow: <b id="p3flow">--</b> L/min</div>
 </div>

 <div class="card">
  <div class="label">SOLENOID VALVE</div>
  <div class="value" id="valve">--</div>
  <div class="small">Case 1 inlet control</div>
 </div>

 <div class="card">
  <div class="label">SUMP FLOAT SWITCH</div>
  <div class="value" id="float">--</div>
  <div class="small">Completely up / sump full status</div>
 </div>

 <div class="card">
  <div class="label">FT01 FLOW RATE</div>
  <div class="value blue"><span id="flow">--</span> L/min</div>
  <div class="small">Calculated from measured pulse rate</div>
 </div>
```

```
 <div class="card">
  <div class="label">TOTAL VOLUME</div>
  <div class="value blue"><span id="litres">--</span> L</div>
  <div class="small">Case 1 target: 7.00 L</div>
 </div>

 <div class="card">
  <div class="label">FT01 PULSES</div>
  <div class="value" id="pulses">--</div>
  <div class="small">Pulses per minute: <b id="ppm">--</b></div>
 </div>

 <div class="card">
  <div class="label">FT01 CALIBRATION</div>
  <div class="value" id="ppl">--</div>
  <div class="small">Pulses per litre (set in code)</div>
 </div>

 <div class="card wide">
  <div class="label">FLUSH ESTIMATE</div>
  <div class="value"><span id="flush">--</span> / 4.00 L</div>
  <div class="small">Pump 3 calibration-based estimate only. It does not physically detect an empty
tank.</div>
 </div>
</div>
</main>
<footer>Fluid Mechanics Lab • Live ESP32 Wi-Fi dashboard • Refreshes every second</footer>
<script>
async function refreshData(){
 try{
  const r=await fetch('/data');
  const d=await r.json();

  document.getElementById('state').textContent=d.state;
  document.getElementById('timer').textContent=d.timer;
  document.getElementById('p1').textContent=d.p1;
  document.getElementById('p2').textContent=d.p2;
  document.getElementById('p3').textContent=d.p3;
  document.getElementById('p1pwm').textContent=d.p1pwm;
  document.getElementById('p2pwm').textContent=d.p2pwm;
  document.getElementById('p3pwm').textContent=d.p3pwm;
  document.getElementById('p1flow').textContent=d.p1flow;
  document.getElementById('p2flow').textContent=d.p2flow;
  document.getElementById('p3flow').textContent=d.p3flow;
  document.getElementById('valve').textContent=d.valve;
  document.getElementById('float').textContent=d.float;
  document.getElementById('flow').textContent=d.flow;
  document.getElementById('litres').textContent=d.litres;
  document.getElementById('pulses').textContent=d.pulses;
  document.getElementById('ppm').textContent=d.ppm;
  document.getElementById('ppl').textContent=d.ppl;
  document.getElementById('flush').textContent=d.flush;

  const pct=Math.min(100,Math.max(0,Number(d.litres)/7*100));
  document.getElementById('progress').style.width=pct+'%';
 }catch(e){}
}
```

```
refreshData();
setInterval(refreshData,1000);
</script>
</body>
</html>
)HTML";

// ====================================================
// DASHBOARD DATA ENDPOINT
// ====================================================
void handleData() {
  unsigned long pulses = getPulseCount();

  int p1pwm = 0, p2pwm = 0, p3pwm = 0;

  if (pump1On) {
    p1pwm = (state == CASE2_RUNNING || state == FINAL_SHUTDOWN)
          ? CASE2_P1_PWM : CASE1_P1_PWM;
  }

  if (pump2On) {
    p2pwm = (state == CASE2_RUNNING || state == FINAL_SHUTDOWN)
          ? CASE2_P2_PWM : CASE1_P2_PWM;
  }

  if (pump3On) p3pwm = CASE2_P3_PWM;

  float p1flow = estimatedPump1Flow(p1pwm);
  float p2flow = estimatedPump2Flow(p2pwm);
  float p3flow = estimatedPump3Flow(p3pwm);

  unsigned long flushSeconds = remainingSeconds();

  String json = "{";
  json += "\"state\":\"" + stateName() + "\",";
  json += "\"timer\":" + String(flushSeconds) + ",";
  json += "\"p1\":\"" + onOff(pump1On) + "\",";
  json += "\"p2\":\"" + onOff(pump2On) + "\",";
  json += "\"p3\":\"" + onOff(pump3On) + "\",";
  json += "\"p1pwm\":" + String(p1pwm) + ",";
  json += "\"p2pwm\":" + String(p2pwm) + ",";
  json += "\"p3pwm\":" + String(p3pwm) + ",";
  json += "\"p1flow\":" + String(p1flow, 2) + ",";
  json += "\"p2flow\":" + String(p2flow, 2) + ",";
  json += "\"p3flow\":" + String(p3flow, 2) + ",";
  json += "\"valve\":\"" + valveText() + "\",";
  json += "\"float\":\"";
  json += sumpFloatCompletelyUp() ? "UP / FULL" : "DOWN / NOT FULL";
  json += "\",";
  json += "\"flow\":" + String(flowLpm, 3) + ",";
  json += "\"litres\":" + String(totalLitres, 3) + ",";
  json += "\"pulses\":" + String(pulses) + ",";
  json += "\"ppm\":" + String(pulsesPerMinute, 1) + ",";
  json += "\"ppl\":" + String(FT01_PULSES_PER_LITRE, 1) + ",";
  json += "\"flush\":" + String(
     (state == CASE2_RUNNING || state == FINAL_SHUTDOWN)
       ? min(FLUSH_TARGET_LITRES,
```

```
            estimatedPump3Flow(CASE2_P3_PWM) *
            ((millis() - case2StartMs) / 60000.0))
        : 0.0, 2);
  json += "}";

  server.send(200, "application/json", json);
}

void handleRoot() {
  server.send_P(200, "text/html", PAGE);
}

// =======================================================
// SETUP
// =======================================================
void setup() {
  Serial.begin(115200);

  configurePump(P1_RPWM, P1_LPWM, P1_REN, P1_LEN);
  configurePump(P2_RPWM, P2_LPWM, P2_REN, P2_LEN);
  configurePump(P3_RPWM, P3_LPWM, P3_REN, P3_LEN);

  pinMode(SOLENOID_PIN, OUTPUT);
  solenoidSet(false);

  // GPIO35 is input-only and has no internal pull-up.
  pinMode(FLOAT_PIN, INPUT);

  pinMode(START_STOP_PIN, INPUT_PULLUP);

  pinMode(FT01_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FT01_PIN),
                  onFlowPulse, RISING);

  allOutputsOff();

  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_SSID, AP_PASSWORD);

  server.on("/", handleRoot);
  server.on("/data", handleData);
  server.begin();

  Serial.println();
  Serial.println("FLUID MECHANICS CONTROLLER READY");
  Serial.print("Wi-Fi network: ");
  Serial.println(AP_SSID);
  Serial.print("Dashboard IP: ");
  Serial.println(WiFi.softAPIP());
  Serial.println("Connect laptop to Wi-Fi and open http://192.168.4.1");
  Serial.println("Press the physical button to start or stop.");
}

// =======================================================
// MAIN LOOP
// =======================================================
void loop() {
```

```
  server.handleClient();
  handleButton();
  updateFlowMeasurements();
  runProcess();
}
```